



Topic:

Jira Framework — Story → Manual + Automated Test Cases

Scope:

The Jira Framework is the requirements-to-test-artefact bridge of the platform. For any Jira issue key (e.g. `SCRUM-5`) — or a local Markdown story — it fetches the story via the Jira REST API v3, converts Atlassian Document Format (ADF) → Markdown, lints the structure (requires `## Story` + `## Acceptance Criteria` with ≥ 2 items), and sends it to Anthropic Claude with the base prompt + AI-agent docs as system context. Output: per-story Excel sheets `Test_Case_<STORY-ID>` in `Manual_Test_Cases.xlsx`. With a locators file present, two more Claude agents generate a TypeScript Page Object (extending `BasePage`) + a Playwright spec. After execution, results are written back to Excel, pushed to Zephyr, and the Jira issue is transitioned to Done. `Pipeline_Config.xlsx` drives multi-story batch runs.

Key Features of the Jira Framework:

- Three-Stage Pipeline:** Story Ingestion → Test Case Generation → Optional Playwright Automation — invoked individually or end-to-end.
- Native Jira Integration:** `fetch-jira.js` calls `GET /rest/api/3/issue/<KEY>`; `jiraToMarkdown.js` flattens ADF to clean Markdown saved to `stories/jira_stories/`.
- Fail-Fast Lint:** `storyLinter.js` validates structure BEFORE any LLM call — no token spend on a malformed story.
- Strict JSON Validation:** `validator.js` enforces priority $\in$ {High, Medium, Low}, type $\in$ {Positive, Negative, Edge}; 3 retries on bad JSON; every LLM output timestamped to `output/json/` before Excel write.
- Three LLM Agents:** `testCaseAgent` (story → cases JSON), `pageObjectAgent` (locators + cases → TS class extending `BasePage`), `playwrightAgent` (cases + page object → `tests/<StoryId>.spec.ts`).
- Per-Story Excel Sheets:** Each story gets its own sheet `Test_Case_<STORY-ID>`; re-runs replace just that sheet — other stories' sheets are untouched.
- Result Writeback:** `testResultsService.js` fills the Automation Status column (Passed / Failed / Timed Out / Skipped) per row from `test-results.json`.
- Zephyr + Jira Closure:** `zephyrService.js` pushes results to the QA test cycle; story is auto-moved to `stories/completed/`; Jira issue transitions to the status configured by `JIRA_DONE_STATUS`.
- Batch Pipeline:** `Pipeline_Config.xlsx` drives `run-pipeline.js` — per-row Story ID, Run Mode (manual / automation / both), Browser, Headless.
- Generic Target:** Any Jira project + any web app — switch via `JIRA_PROJECT_KEY` + `BASE_URL`.

Why Use the Jira Framework?

Requirements live in Jira; manual cases live in Excel; automation lives in Playwright; results live in Zephyr. Keeping these in sync by hand is the single biggest source of slip in agile QA. The Jira Framework collapses the chain: one command takes a story key and produces all four artefacts plus a status writeback. The story is the only thing a human writes — every downstream artefact regenerates from it. Excel sheet replacement is surgical, so re-running one story never disturbs others; the locators-file requirement keeps generated page objects deterministic so re-generation never invents selectors that don't exist on the page.

`Pipeline_Config.xlsx` turns the same flow into a batch driver for whole sprints.

Jira Framework Architecture



Integration with Master Orchestrator, Tooling & CI/CD

Invoked via `node index.js --framework=jira`; `jiraAdapter.js` runs generation + automation back-to-back in the framework's serial slot. Built on **Node.js** + **Anthropic Claude (Sonnet 4.6)** + **Jira REST API v3** + **Zephyr** + **Playwright** + **TypeScript** + **BasePage pattern** + **ExcelJS**. Required `.env`: `JIRA_BASE_URL`, `JIRA_EMAIL`, `JIRA_API_TOKEN`, `JIRA_PROJECT_KEY`, `JIRA_DONE_STATUS`, `BASE_URL`, `TEST_USERNAME`, `TEST_PASSWORD`, `HEADLESS`, `WORKERS`, `ZEPHYR_TOKEN`. Playwright reporters (HTML + JSON) plug directly into Jenkins / GitHub Actions / Azure DevOps; the JSON reporter is what writes back to Excel.

